

STRIDE Analyzer

Infográfico — Diagrama de Integração

Python · Docker · Kaggle · OpenAI GPT-4o Vision

O projeto integra módulos Python e contêineres Docker a serviços externos via variáveis de ambiente (.env)

SERVIÇOS EXTERNOS (backing services — Twelve-Factor IV)

Kaggle API

Dataset: carlosrian/software-architecture-dataset
Credenciais: KAGGLE_USERNAME + KAGGLE_KEY
Usado por: dataset.py / serviço download
Protocolo: HTTPS (CLI kaggle download --unzip)

OpenAI GPT-4o Vision

Modelo: OPENAI_MODEL (padrão gpt-4o)
Credencial: OPENAI_API_KEY
Usado por: stride.py / serviço analyze
Entrada: imagem PNG em base64 + prompt STRIDE

Hugging Face (CLIP)

Modelo zero-shot (transformers + torch)
Cache: volume hf-cache
Usado por: image_analysis.py / filter
Classifica diagramas de arquitetura

AMBIENTES DE EXECUÇÃO

A) Execução local (venv + CLI)

Pacote Python stride-analyzer

pip install -e ".[dev]"
comando: stride-analyzer <setup|download|filter|analyze|run-all>

cli.py

config.py

dataset.py

filter.py

image_analysis.py

stride.py

reports.py

paths.py

.env → Settings.from_env() → módulos → pastas locais

B) Execução Docker Compose

Imagem Dockerfile (python:3.11-slim)

entrypoint → stride-analyzer
profiles: modular | monolith

setup

pastas/volumes

download

→ Kaggle

filter

→ CLIP

analyze

→ OpenAI

pipeline

run-all

volumes: dataset · diagramas · relatório · hf-cache

FLUXO DE INTEGRAÇÃO Ponta a Ponta

1. Credenciais

.env
Kaggle + OpenAI

2. Download

Kaggle API
→ dataset/

3. Filter

OpenCV + CLIP
→ diagramas/

4. Analyze

GPT-4o Vision
→ ameaças STRIDE

5. Reports

JSON · PDF · DOCX
→ relatório/

Quem chama o quê

Módulo / Serviço	Serviço externo	Dados trocados	Config
dataset.py / download	Kaggle	ZIP/imagens de arquitetura	KAGGLE_*
image_analysis.py / filter	Hugging Face CLIP	rótulos zero-shot	HF cache
stride.py / analyze	OpenAI GPT-4o Vision	PNG base64 + texto STRIDE	OPENAI_*
reports.py / analyze	(local)	arquivos JSON/PDF/DOCX	relatório/

CONTRATO DE INTEGRAÇÃO E SEGURANÇA

- Config centralizada**
Todas as chaves vêm do .env (nunca commitado). Docker Compose usa env_file opcional.
- Kaggle**
download é idempotente: se dataset/ já tem imagens, não chama a API novamente.
- OpenAI**
cada PNG selecionado vira uma chamada Vision; custo/latência dependem de MAX_IMAGENS.
- CLIP**
primeira execução baixa o modelo; depois reutiliza o volume hf-cache no Docker.
- Paridade local/Docker**
mesma CLI e mesma Settings — só muda o runtime (host vs contêiner).
- Saída**
artefatos ficam no host via bind mount (/relatorio) ou pastas locais no venv.

Legenda de cores: Azul Docker · Ciano Kaggle · Verde OpenAI · Laranja CLIP · Roxo Relatórios